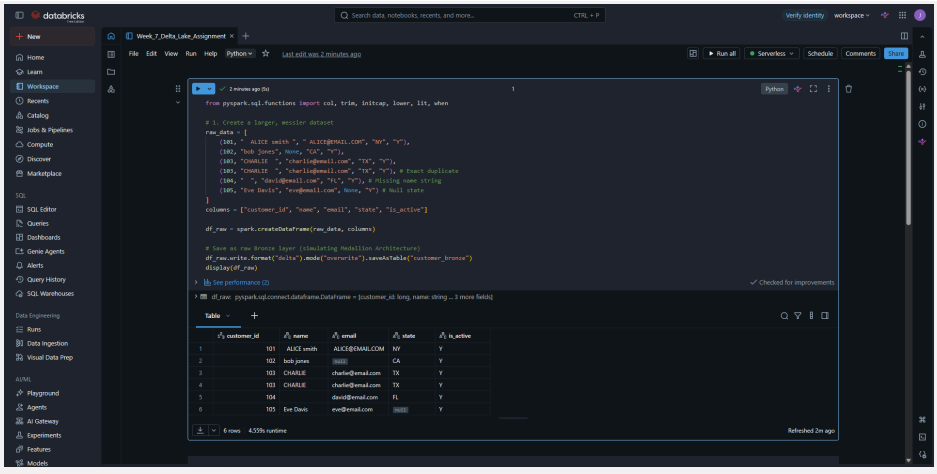
Celebal Week 7 Assignment

Objective:
To design and implement a robust data ingestion and cleaning pipeline using Apache Spark, culminating in a Delta Lake MERGE operation to handle incremental data updates.

1. Data Loading (Bronze Layer)

To kick off the pipeline, I put together a messy, raw dataset to mimic real-world incoming source records. I intentionally baked in all the classic data quality headaches, things like annoying leading and trailing whitespaces, all over the place capitalization, missing values, nulls, and duplicate entries.

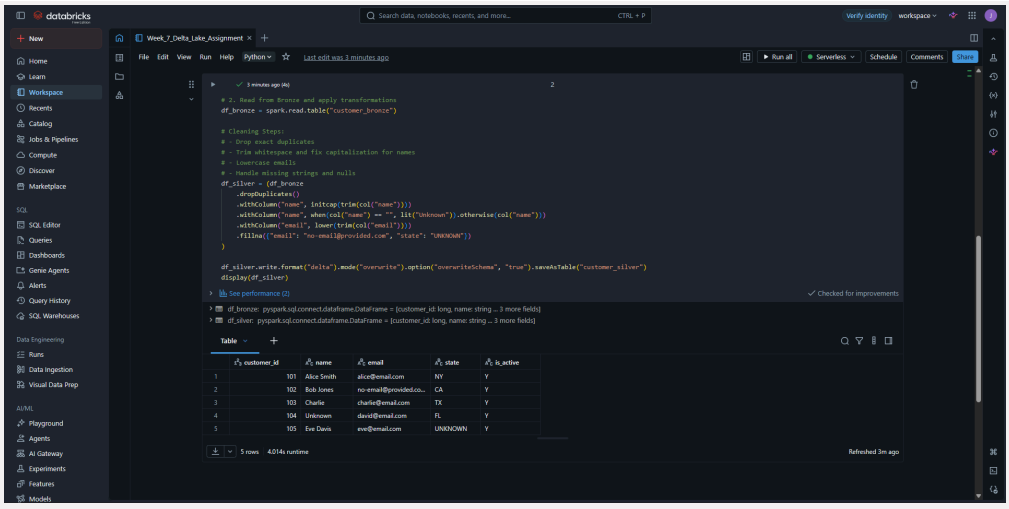
From there, I saved everything straight into Databricks using the default Delta Lake format. Under the hood, a Delta table pairs standard Parquet files with a `_delta_log` directory full of JSON files. That transaction log is what gives you ACID compliance and handles concurrent writes safely, stuff you just don't get with plain Parquet or Hive tables. Sticking to a Medallion Architecture, this raw, completely untouched data forms our Bronze layer, making sure we keep the original state fully intact for data lineage and auditing down the road.



2. Data Cleaning & Structuring (Silver Layer)

Following the multi-hop architecture pattern, the data was progressively refined from the Bronze layer into a cleaner, enterprise-wide Silver layer.
The following PySpark transformations were applied to structure the data:

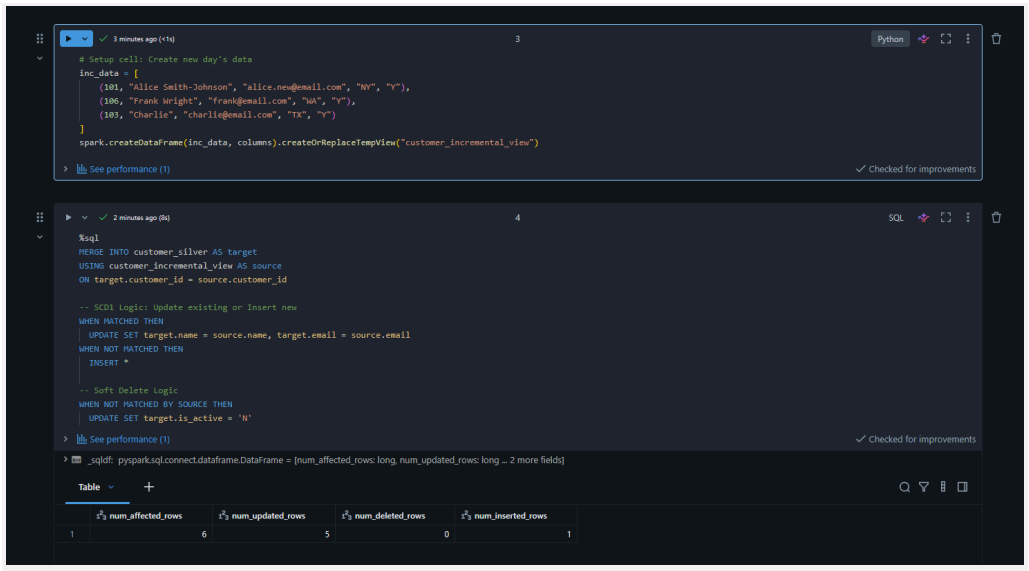
- Deduplication: Dropped identical rows to ensure data integrity.
- String Manipulation: Applied `trim()` and `initcap()` to standardize names, and `lower()` to standardize email addresses.
- Null Handling: Replaced empty strings with a default "Unknown" string and used `.fillna()` to provide default values for missing emails and states.



3. Incremental Upserts (SCD1) & The MERGE Operation

To test the pipeline's ability to handle evolving data over time, a second dataset (customer_incremental) was introduced representing a new day's data.
A SQL MERGE statement was used to upsert this incremental data into the target Silver table. The operation relied on up to three distinct matching clauses to determine the outcome of each row:

- WHEN MATCHED: Implemented Slowly Changing Dimension Type 1 (SCD1) logic by updating the target row's email and name with the new values from the source, intentionally overwriting the old state with no history kept.
- WHEN NOT MATCHED: Inserted entirely new customer records that only existed in the source data.



4. Historical Preservation via Soft Deletes

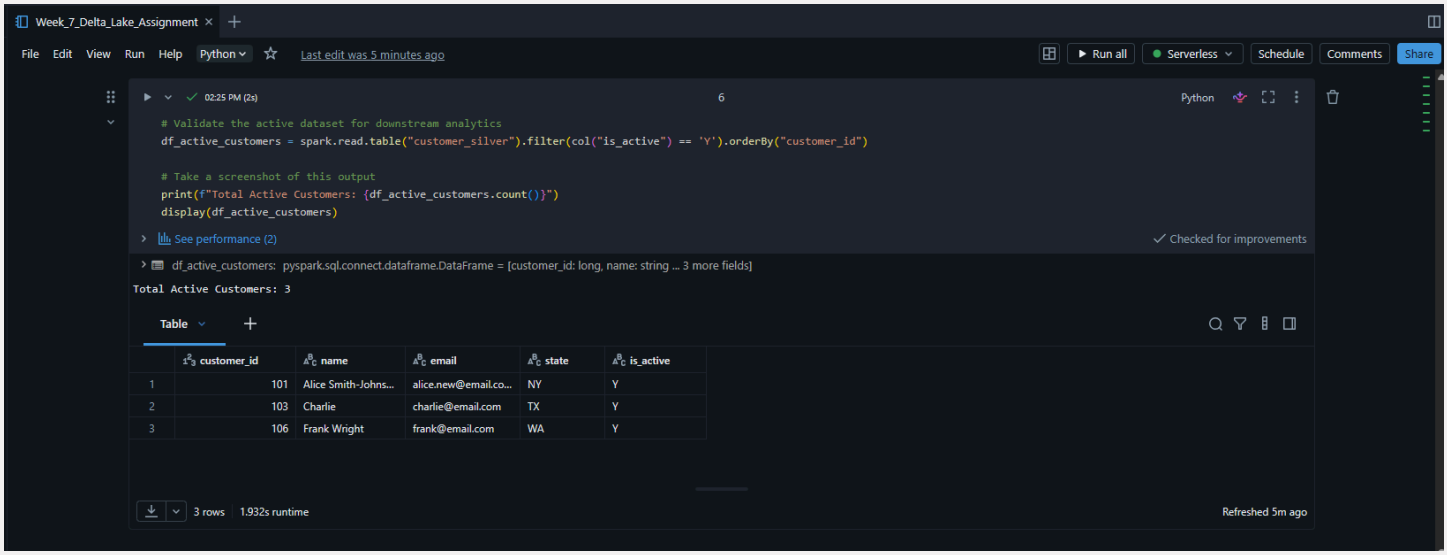
Instead of executing a hard DELETE for records that dropped out of the source system, the MERGE operation utilized the third matching clause.

- WHEN NOT MATCHED BY SOURCE: Handled keys that were present in the target but absent from the source.

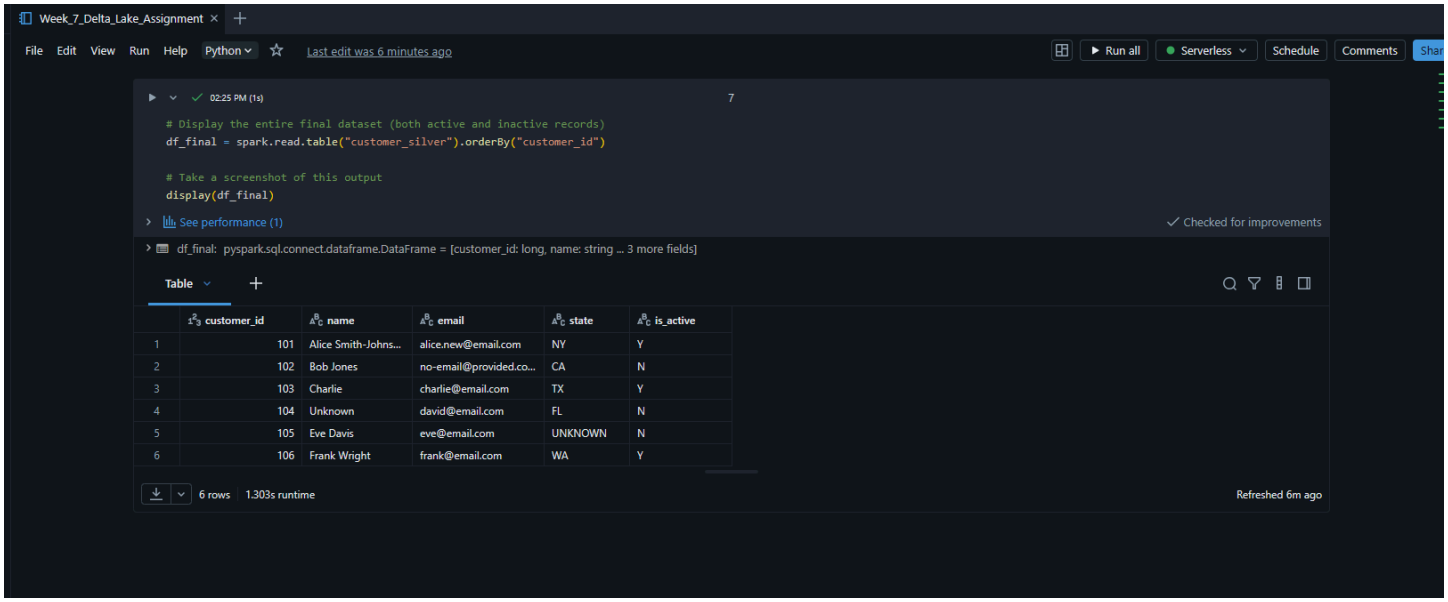
This clause was used to implement a soft delete by updating an `is_active` flag to 'N' instead of physically erasing the row. This approach preserves historical data and ensures past records remain fully queryable for auditing purposes.

5. Validation and Final Output

To confirm the success of the pipeline, the dataset was queried to validate the downstream impact. The current, active state of the table can easily be accessed by analysts simply by filtering for `is_active = 'Y'`.



The final dataset accurately reflects the updated information for existing keys, the insertion of the new keys, and the successful application of the 'N' flag for stale records.


Week_7_Delta_Lake_Assignment
File Edit View Run Help Python
Last edit was 6 minutes ago
Run all
Serverless
Schedule
Comments
02:25 PM (1s)
7
# Display the entire final dataset (both active and inactive records)
df_final = spark.read.table("customer_silver").orderBy("customer_id")
# Take a screenshot of this output
display(df_final)
See performance (1)
Checked for improvements
df_final: pyspark.sql.connect.dataframe.DataFrame = [customer_id: long, name: string ... 3 more fields]
Table
customer_id name email state is_active
1 101 Alice Smith-Johns... alice.new@email.com NY Y
2 102 Bob Jones no-email@provided.co... CA N
3 103 Charlie charlie@email.com TX Y
4 104 Unknown david@email.com FL N
5 105 Eve Davis eve@email.com UNKNOWN N
6 106 Frank Wright frank@email.com WA Y
6 rows 1.303s runtime
Refreshed 6m ago